



Project name: University Information System

Discipline: Object-Oriented Programming

Team members:
Bakhramov Abdulaziz
Saden Daniyar
Kairbek Abulhair
Abduova Akbota

The main goal of the project:

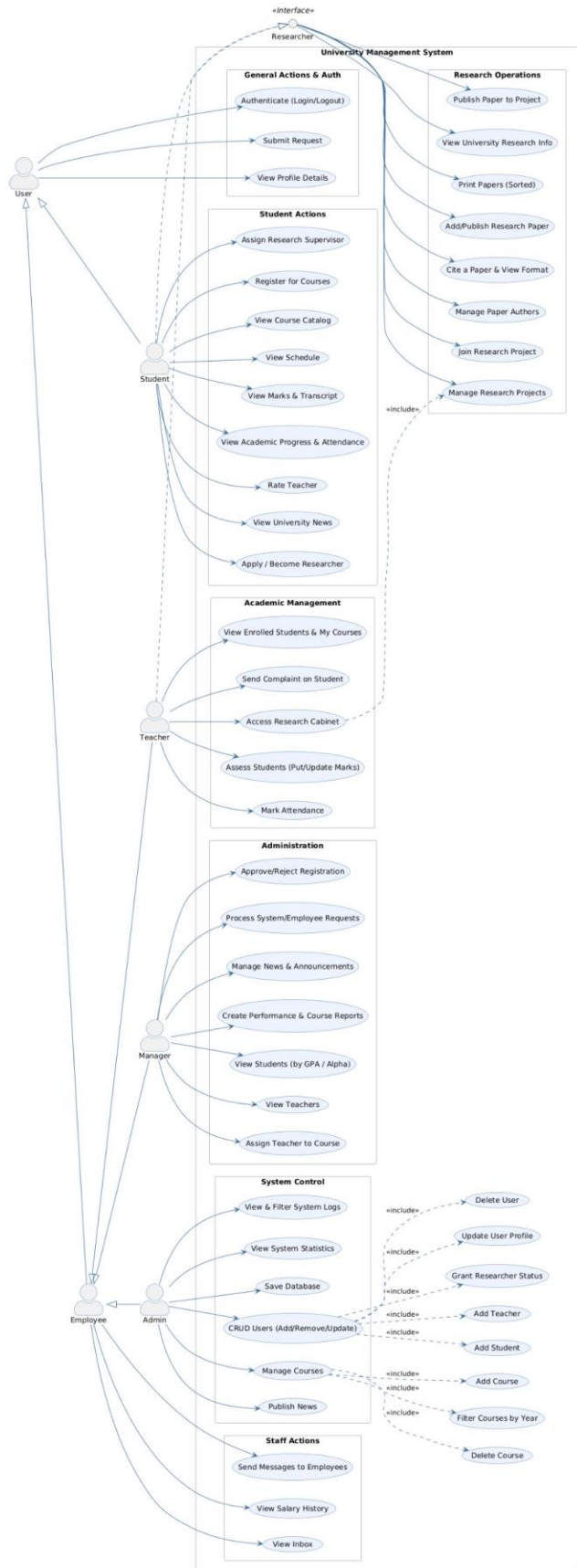
Build a console Java application — an information system for a research university. The system covers course registration, grading, research paper tracking, and access for different user roles.

The main purpose was to practice everything from the OOP course: inheritance, interfaces, abstract classes, exceptions, enums, design patterns, and serialization — all in one real working project.

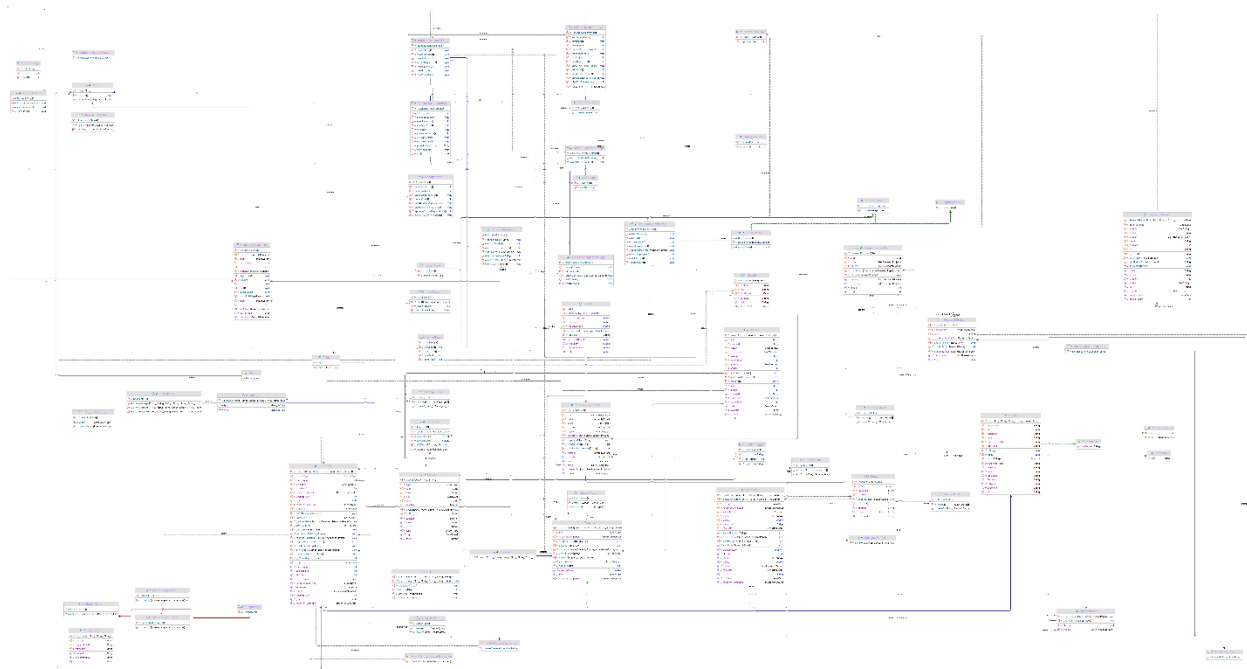
Project description:

- Design Use Case and UML Class Diagrams in StarUML
- Implement all classes in Java using Eclipse IDE
- Generate Javadoc documentation
- Write report and prepare presentation

1. Diagrams**Use case Diagram:**



Class Diagram:



The diagram shows the class hierarchy. User is abstract at the top. Employee extends User, and Teacher, Manager, Admin extend Employee. Student extends User directly. Professor is a subclass of Teacher. IResearcher is an interface — any class can get research functionality through ResearchDecorator.

2. Working with Eclipse

Packages

Classes are split into 10 packages:

- users — User, Employee, Student, Teacher, Professor, Manager, Admin, UserFactory, enums
- academic — Course, Mark, Lesson, Transcript, enums (LessonType, AttestationType, etc.)
- research — IResearcher, ResearchDecorator, ResearchPaper, ResearchProject
- exceptions — UniversityException, CreditLimitExceededException, IndexTooLowException, NonResearcherException
- core — UniversityKernel, SystemLogger, interfaces
- utils — DataStorage, Comparators, InputSystem, ConsoleRenderer
- console — page classes, UniversityConsole
- parsers — IInputParser, IntegerParser, StringParser, DoubleParser
- infrastructure — NewsEntry, Request, Registration
- application — Main

User class

Abstract root of the whole hierarchy. Implements IIdentifiable, IAuthenticatable, ILoggable, and Serializable:

```
public abstract class User implements IIdentifiable, ILoggable, IAuthenticatable, Serializable {
    protected String id, username, passwordHash, firstName, lastName, email;

    @Override
    public boolean login(String credentials) {
        return passwordHash.equals(credentials);
    }

    @Override
    public String toString() {
        return getClass().getSimpleName() + " [id=" + id + ", firstName=" + firstName + ", lastName=" + lastName + "];"
    }
}
```

Student class

Extends User. Represents a bachelor student. Main fields:

```
private DegreeType degreeType;
private int yearOfStudy;
private int totalCredits;
private int failedCount;
private Transcript transcript;
private List<Course> currentCourses;
private IResearcher researchSupervisor;
```

registerForCourse() checks the 30-credit limit and blocks if student failed more than 3 times:

```
public boolean registerForCourse(Course c) throws CreditLimitExceededException {
    if (totalCredits + c.getCredits() > 30) {
        throw new CreditLimitExceededException("Credit limit exceeded (max 30 credits)");
    }
    if (failedCount > 3) {
        throw new RuntimeException("Cannot register for courses: you have failed more than 3 times.");
    }
    currentCourses.add(c);
    totalCredits += c.getCredits();
    return true;
}
```

4th-year students can get a research supervisor. If supervisor h-index < 3, IndexTooLowException is thrown:

```
public void setResearchSupervisor(IResearcher supervisor) throws IndexTooLowException {
    if (supervisor != null && supervisor.getHIndex() < 3) {
        throw new IndexTooLowException("Supervisor h-index must be at least 3 for " + getFullName());
    }
    this.researchSupervisor = supervisor;
}
```

Teacher class

Extends Employee. Has TeacherTitle enum (TUTOR, LECTURER, PROFESSOR). When title is PROFESSOR, ResearchDecorator is attached automatically in the constructor:

```
public Teacher(String id, String username, String passwordHash, String firstName,
                String lastName, String email, double salary,
                LocalDate hireDate, String department, TeacherTitle title) {
    super(id, username, passwordHash, firstName, lastName, email, salary, hireDate, department);
    this.title = title;
    this.courses = new ArrayList<>();
    if (title == TeacherTitle.PROFESSOR) {
        this.researchComponent = new ResearchDecorator(this);
    }
}
```

putMark() and viewStudents():

```
public void putMark(Student s, Course c, Mark m) {
    s.addMark(c, m);
    System.out.println("Mark put for student " + s.getFullName() + " in course " + c.getName());
}

public List<Student> viewStudents(String courseId) {
    return UniversityKernel.getInstance().getUsers().stream()
        .filter(u -> u instanceof Student)
        .map(u -> (Student) u)
        .filter(s -> s.viewCourses().stream()
            .anyMatch(c -> c.getCourseId().equals(courseId)))
        .collect(java.util.stream.Collectors.toList());
}
```

Mark class

Three components: firstAttestation, secondAttestation, finalExam. setMark() uses AttestationType enum:

```
public class Mark implements Serializable {
    private double firstAttestation;
    private double secondAttestation;
    private double finalExam;

    public void setMark(AttestationType type, double value) {
        switch (type) {
            case FIRST_ATT: firstAttestation = value; break;
            case SECOND_ATT: secondAttestation = value; break;
            case FINAL_EXAM: finalExam = value; break;
        }
    }

    public double getTotal() {
        return firstAttestation + secondAttestation + finalExam;
    }
}
```

Research module

IResearcher is an interface. ResearchDecorator wraps any User and adds paper management and h-index calculation:

```
public int getHIndex() {
    if (researchPapersList.isEmpty()) {
        return 0;
    }
    List<Integer> citations = researchPapersList.stream()
        .map(ResearchPaper::getCitations)
        .sorted(Comparator.reverseOrder())
        .collect(Collectors.toList());
    int hIndexValue = 0;
    while (hIndexValue < citations.size() && citations.get(hIndexValue) >= hIndexValue + 1) {
        hIndexValue++;
    }
    return hIndexValue;
}
```

ResearchProject.addParticipant() throws NonResearcherException if argument is null or already in the list:

```
public void addParticipant(IResearcher r) throws NonResearcherException {
    if (r == null) {
        throw new NonResearcherException("Participant must be a researcher");
    }
    if (participants.contains(r)) {
        throw new NonResearcherException("You are already a participant in this project");
    }
    participants.add(r);
}
```

Design Patterns

We used 4 patterns:

- Singleton — UniversityKernel, one instance for the whole app
- Decorator — ResearchDecorator, adds research to any User without changing its class
- Factory Method — UserFactory, creates Student/Teacher/Manager with one call
- Strategy — Comparators, swap sorting logic for research papers (by date, citations, pages)

Data persistence (Serialization)

All data is saved through DataStorage using Java serialization. Called on startup and on exit:

```
public static void save() {
    File dir = new File("database");
    if (!dir.exists()) dir.mkdirs();
    try (ObjectOutputStream oos = new ObjectOutputStream(
        new FileOutputStream(new File(dir, "data")))) {
        oos.writeObject(UniversityKernel.getInstance());
    } catch (IOException e) {
    }
}
```

Console interface

After login, UniversityConsole routes the user to their page based on role:

Student: view courses, register, view marks, transcript, rate teachers, schedule

Teacher: manage courses, put marks, attendance, view students, complaints

Manager: approve registrations, assign courses, statistics, news

Admin: add/remove/update users, view log

Research cabinet: add papers, print sorted by date/citations/pages, manage projects

3. Documentation

Javadoc was generated through Eclipse for the main classes: User, Student, Teacher, Employee, Course, Mark, ResearchPaper, ResearchDecorator, UniversityKernel, DataStorage.

The screenshot displays the Javadoc documentation for the `Student` class. At the top, there is a navigation bar with links for Overview, Package, Class, Tree, Index, and Help. Below this, the package is listed as `com`. The class `Student` is shown as extending `User` and implementing `Researcher`. A list of implemented interfaces includes `Serializable`, `Identifiable`, `Runnable`, `Serializable`, and `Researcher`. The `Student` class is defined as `public class Student extends User implements Researcher`. A section titled "Field Summary" lists fields inherited from `User`: `email`, `firstName`, `id`, `lastName`, `passwordHash`, and `username`. The "Constructor Summary" section shows a constructor `Student(String id, String username, String passwordHash, String firstName, String lastName, String email, DegreeType degreeType, int yearOfStudy)`. The "Method Summary" section is divided into three tabs: All Methods, Instance Methods, and Concrete Methods. The "All Methods" tab is selected, showing a table of methods with their modifiers, types, names, and descriptions. The methods include `addAttendance`, `addCourseSchedule`, `addMark`, `addPaper`, `getAttendance`, `getAttendanceRecords`, `getDegreeType`, `getEnrollment`, `getIndex`, `getPapers`, `getResearchComponent`, `getResearchSupervisor`, `getSchedule`, `getTotalCredits`, `getTranscript`, `getYearOfStudy`, `printPapers`, and `rateTeacher`.

Modifier and Type	Method	Description
void	<code>addAttendance(Course c)</code>	
void	<code>addCourseSchedule(Course c)</code>	
void	<code>addMark(Course c, Mark m)</code>	
void	<code>addPaper(ResearchPaper p)</code>	
int	<code>getAttendance(Course c)</code>	
List<AttendanceStatus>	<code>getAttendanceRecords(Course c)</code>	
DegreeType	<code>getDegreeType()</code>	
int	<code>getEnrollment()</code>	
double	<code>getIndex()</code>	
List<ResearchPaper>	<code>getPapers()</code>	
ResearchDecorator	<code>getResearchComponent()</code>	
Researcher	<code>getResearchSupervisor()</code>	
List<Lesson>	<code>getSchedule()</code>	
int	<code>getTotalCredits()</code>	
Transcript	<code>getTranscript()</code>	
int	<code>getYearOfStudy()</code>	
void	<code>printPapers(Comparator<ResearchPaper> sorter)</code>	
void	<code>rateTeacher(Teacher t for course)</code>	

OVERVIEW

PACKAGE

CLASSES

TREE

INDEX

HELP

SUMMARY NESTED (FIELD) CONSTR. METHOD DETAIL FIELD CONSTR. METHOD

SEARCH

Constructor Summary

Constructors

Constructor

Student(String id, String username, String passwordHash, String firstName, String lastName, String email, DegreeType degreeType, int yearOfStudy)

Description

Method Summary

All Methods

Instance Methods

Concrete Methods

Modifier and Type

Method

Description

void

addAttendance(course c)

void

addCourseSchedule(course c)

void

addMark(course c, Mark m)

void

addPaper(ResearchPaper p)

int

getAttendance(course c)

List<AttendanceStatus>

getAttendanceRecords(course c)

DegreeType

getDegreeType()

int

getFailCount()

double

getGpa()

int

getIndex()

List<ResearchPaper>

getPapers()

ResearchDecorator

getResearchComponent()

IResearcher

getResearchSupervisor()

List<Lesson>

getSchedule()

int

getTotalCredits()

Transcript

getTranscript()

int

getYearOfStudy()

void

printPapers(Comparator<ResearchPaper> sorter)

void

rateTeacher(teacher t, int score)

void

recordAttendance(course c, AttendanceStatus status)

boolean

registerForCourse(course c)

void

setResearchComponent(ResearchDecorator researchComponent)

void

setResearchSupervisor(IResearcher supervisor)

List<Course>

viewCourses()

List<Mark>

viewMarks()

Methods inherited from class java.lang.Object

getMail, getFirstName, getFullName, getId, getLastName, getIgDetails, getPasswordHash, getUsername, login, logout, setMail, setFirstName, setLastName, setUsername, toString

Methods inherited from class java.lang.Object*

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait*

Constructor Details

Student

Project Management:

We split the work right away — each person took one part of the system. Coordination happened in a Telegram chat and Teams. Most problems came up during integration, when some classes didn't match up because we wrote them separately. Fixed that by agreeing on field names and method signatures together.

Problems:

1. We weren't sure how to model Researcher — interface, abstract class, or just a regular class. Ended up making IResearcher an interface and using ResearchDecorator so any user can become a researcher without touching their class.
2. Serialization gave us trouble — especially the circular references between Course and Teacher. Turns out Java handles it automatically through the object graph, so we didn't need to do anything special.
3. During integration different members had named some fields differently. Took a bit of time to fix, but after that everything compiled fine.